ماشین بردار پشتیبان

به‌همراه مثال‌های کاربردی، توضیحات آماری و کد پایتون

نگارنده: آرمان موجودی

برای دانلود و یا اجرای آنلاین کد پایتون این آموزش به آدرس گیتهاب آموزشها مراجعه کنید.

https://github.com/ML-OilGas/ML_Algorithms/SupportVectorMachine

ارتباط با نگارنده و ارائه‌ی نظرات و پیشنهادات:

https://www.linkedin.com/in/arman-mojoodi

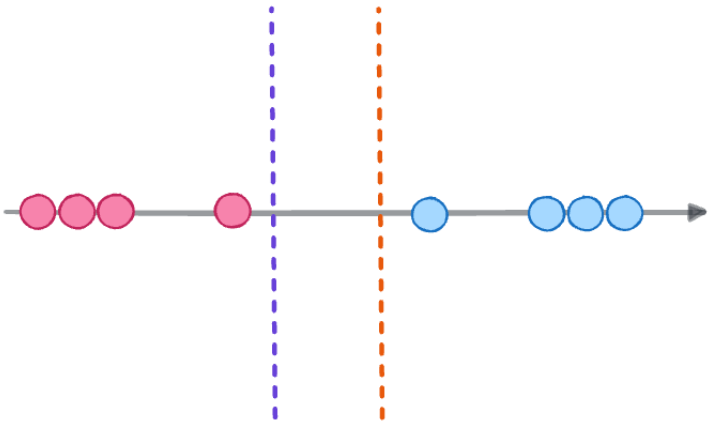
به او...

از صبا پرس که ما را همه شب تا دم صبح

بوی زلف تو همان مونس جان است که بود

مقدمه

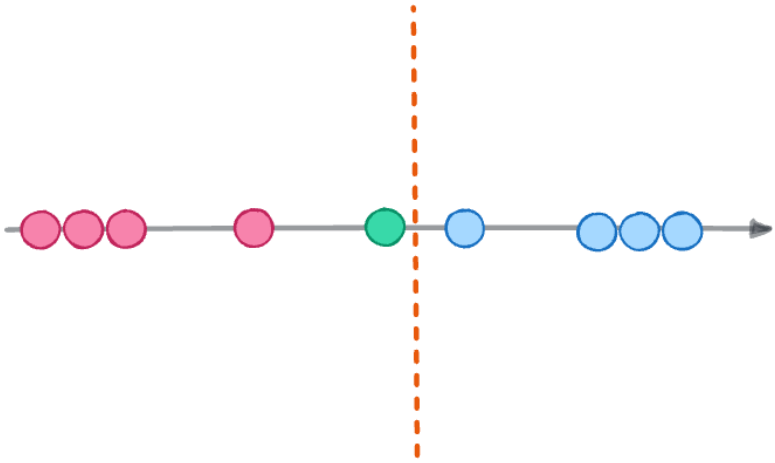
فرض کنید داده‌های زیر در دو دسته‌ی قرمز و آبی قرار دارند و می‌خواهیم آن‌ها را طبقه‌بندی کنیم.



شکل ۱ : دسته‌بندی داده‌ها با مرزهای تصمیم مختلف

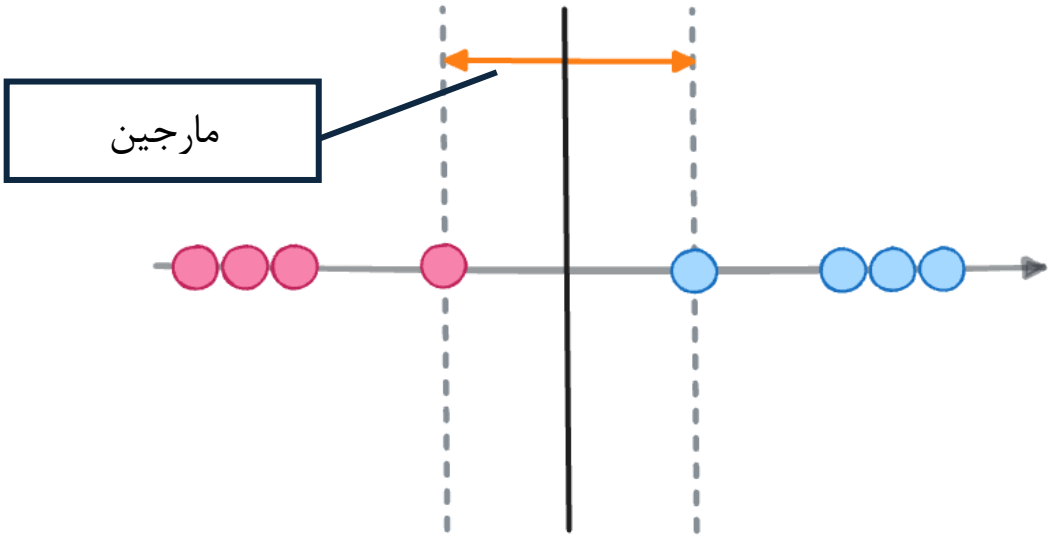
همان‌طور که مشاهده می‌کنید، می‌توان مرزهای مختلفی برای جداسازی این دو دسته رسم کرد؛ از جمله خط‌های بنفش و نارنجی که در شکل ۱ نشان داده شده‌اند. اما انتخاب مرز تصمیم اهمیت زیادی دارد، زیرا مرزهای متفاوت ممکن است به نتایج متفاوتی برای داده‌های جدید منجر شوند.

برای مثال، اگر خط نارنجی را به‌عنوان مرز تصمیم انتخاب کنیم، ممکن است انتظار داشته باشیم داده‌ی جدیدی که با رنگ سبز نشان داده شده است (شکل ۲)، با توجه به موقعیت آن نسبت به داده‌های آموزشی، در دسته‌ی آبی قرار بگیرد؛ اما این مرز، داده‌ی موردنظر را در دسته‌ی قرمز قرار می‌دهد. بنابراین، صرفاً جدا کردن داده‌های آموزشی کافی نیست و محل قرارگیری مرز تصمیم نیز اهمیت دارد.



شکل ۲ : اهمیت مکان مرز تصمیم

یکی از راه‌های انتخاب مرز مناسب، بیشینه‌کردن فاصله‌ی آن از نزدیک‌ترین داده‌های هر دو دسته است. خط مشکی در شکل زیر نمونه‌ای از چنین مرزی است.



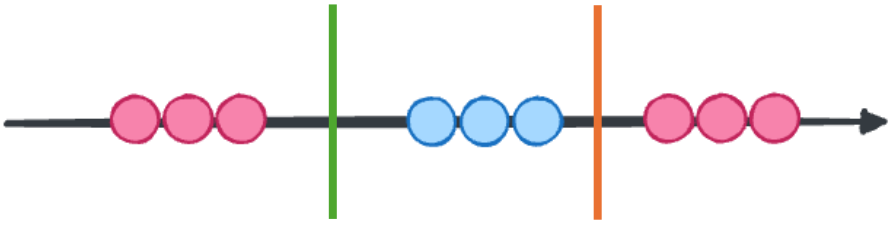
شکل ۳ : طبقه‌بند با بیشینه‌ی حاشیه

به فاصله‌ی مرز تصمیم تا نزدیک‌ترین داده‌های آموزشی، حاشیه[1] گفته می‌شود. مرزی که این فاصله را بیشینه کند، طبقه‌بند با بیشینه‌ی حاشیه[2] نام دارد.

داده‌هایی که نزدیک‌ترین فاصله را با مرز تصمیم دارند و در تعیین موقعیت آن نقش اصلی ایفا می‌کنند، بردارهای پشتیبان[3] نامیده می‌شوند. این داده‌ها در تعیین مرز تصمیم نقش اساسی دارند و تغییر موقعیت آن‌ها می‌تواند بر مرز به‌دست‌آمده تأثیر بگذارد.

جداسازی داده‌های غیرخطی

حال مجموعه‌داده‌ی دیگری را در نظر بگیرید.



شکل ۴ : عدم امکان جداسازی داده‌ها با مرز خطی

---

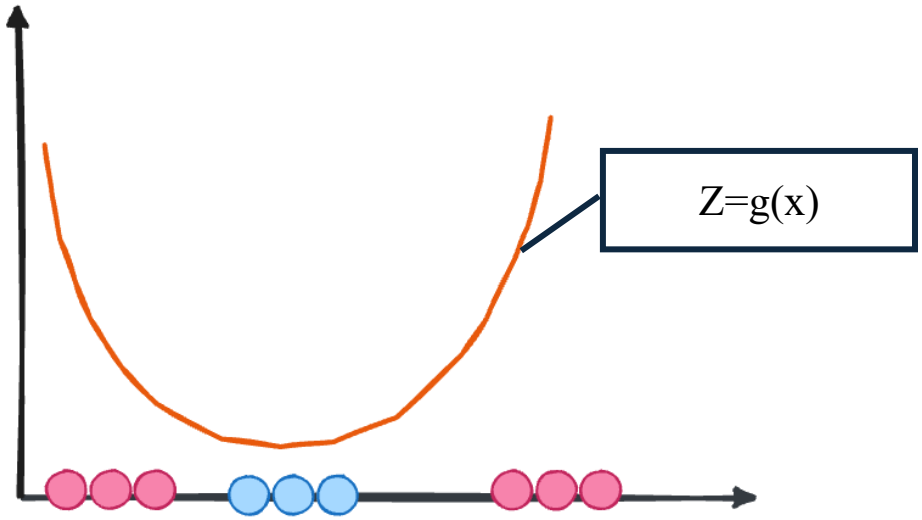
[1] Margin

[2] Maximal Margin Classifier

[3] Support Vectors

همان‌طور که مشاهده می‌کنید، امکان جداسازی کامل دو دسته با یک خط مستقیم وجود ندارد. در این شرایط، انتخاب موقعیت‌های مختلف برای یک مرز خطی نیز نمی‌تواند دو دسته را به‌طور کامل از یکدیگر جدا کند (خط سبز یا نارنجی در شکل ۴).

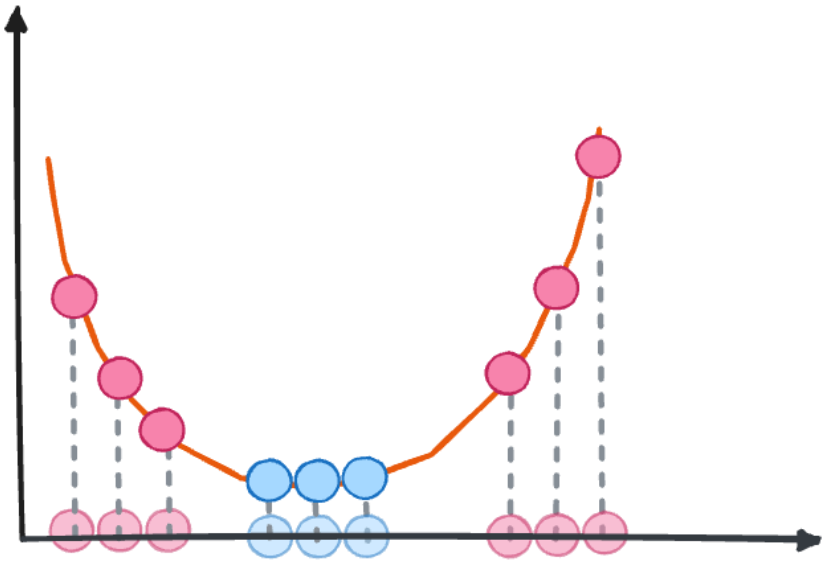
یکی از راه‌های مواجهه با چنین مسئله‌ای، انتقال داده‌ها به فضای ویژگی با ابعاد بالاتر[4] است. برای مثال، داده‌های شکل ۴ را در فضای یک‌بعدی در نظر بگیرید؛ حال فضای دوبعدی جدیدی ایجاد می‌کنیم که در آن رابطه‌ی $Z=g(x)$ که با نمودار سهمی شکل نارنجی نشان داده شده است برقرار باشد (شکل ۵).



شکل ۵ : ساخت فضای ویژگی دوبعدی

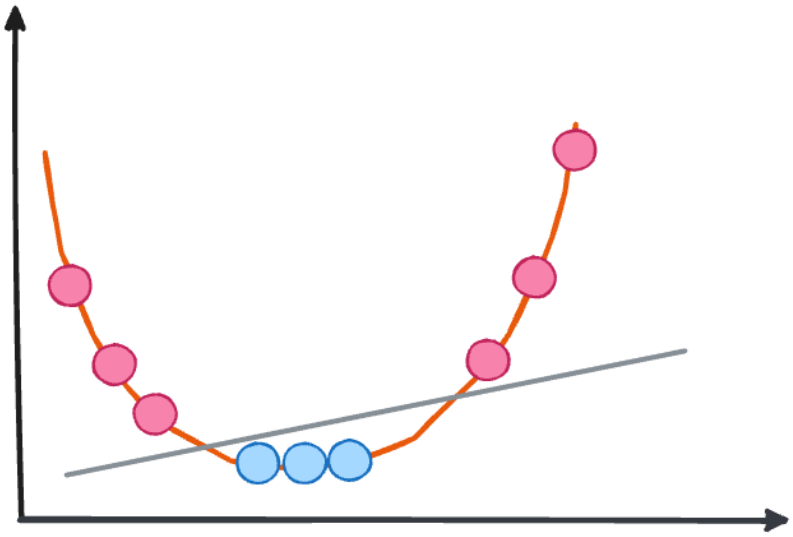
حالا داده‌ها را از فضای یک بعدی قبلی به دوبعدی فعلی تصویر می‌کنیم (شکل ۶). در این حالت، هر داده‌ی اولیه با مختصات جدید $(x, g(x))$ در فضای دوبعدی نمایش داده می‌شود.

[4] Feature Mapping



شکل ۶ : انتقال داده‌ها از فضای یک‌بعدی به دوبعدی

همان‌طور که در شکل ۷ مشاهده می‌کنید، ممکن است داده‌هایی که در فضای اولیه با یک مرز خطی قابل جداسازی نبودند، پس از انتقال به فضای ویژگی جدید، با یک خط مستقیم از یکدیگر جدا شوند.



شکل ۷ : جداسازی داده‌ها با مرز خطی در فضای دوبعدی

این مثال مفهوم کلی نگاشت ویژگی را نشان می‌دهد. در ماشین بردار پشتیبان[5]، برای مدل‌سازی روابط غیرخطی میان داده‌ها، می‌توان از نگاشت ویژگی به فضای با ابعاد بالاتر استفاده کرد؛ به‌طوری‌که داده‌هایی که در فضای اولیه با یک مرز خطی قابل جداسازی نیستند، در فضای جدید با یک مرز خطی جدا شوند.

[5] Support Vector Machine (SVM)

با این حال، محاسبه‌ی صریح مختصات داده‌ها در فضاهای ویژگی با ابعاد بالا ممکن است پیچیده یا پرهزینه باشد. در چنین شرایطی، تابع کرنل[۶] امکان محاسبه‌ی روابط موردنیاز در فضای ویژگی را بدون نیاز به محاسبه‌ی مستقیم نگاشت داده‌ها فراهم می‌کند. از کرنل‌های رایج در الگوریتم SVM می‌توان به کرنل چندجمله‌ای[۷] و کرنل تابع پایه‌ی شعاعی[۸] اشاره کرد.

در مجموع، ایده‌ی اصلی SVM یافتن مرزی مناسب برای جداسازی کلاس‌ها با حاشیه‌ی بزرگ است. این الگوریتم با استفاده از کرنل‌ها می‌تواند روابط پیچیده‌تر میان داده‌ها را نیز مدل‌سازی کند و در مسائل طبقه‌بندی و رگرسیون به کار رود.

در بخش بعد، برای یک دیتاست نفتی واقعی و با استفاده از پایتون، این الگوریتم برای دو مسئله‌ی طبقه‌بندی و رگرسیون استفاده و نتایج آن تحلیل می‌شود.

---

[6] Kernel Function

[7] Polynomial Kernel

[8] Radial Basis Function (RBF)

اجرای عملی

در این بخش، الگوریتم ماشین بردار پشتیبان را روی مجموعه‌داده‌ی Volve که پیش‌تر در بخش رگرسیون خطی چندگانه معرفی شد، پیاده‌سازی می‌کنیم. هدف، بررسی کاربرد این الگوریتم در دو مسئله‌ی متفاوت یادگیری ماشین، یعنی **طبقه‌بندی** و **رگرسیون** است.

ابتدا از طبقه‌بند بردار پشتیبان[9] برای پیش‌بینی کلاس لیتوتایپ‌ها استفاده می‌کنیم. سپس، با استفاده از رگرسیون بردار پشتیبان[10] متغیر هدف را بر اساس متغیرهای ورودی پیش‌بینی کرده و عملکرد مدل را با معیارهای ارزیابی رگرسیون بررسی می‌کنیم.

طبقه‌بندی بردار پشتیبان

در این مثال، از الگوریتم طبقه‌بندی بردار پشتیبان برای پیش‌بینی کلاس لیتوتایپ‌ها در مجموعه‌داده‌ی Volve استفاده می‌کنیم (این مسئله در آموزش رگرسیون لجستیک معرفی شده است). پس از آموزش مدل، پیش‌بینی‌ها روی داده‌های آزمون انجام می‌شوند و عملکرد آن با استفاده از دقت طبقه‌بندی و معیارهای Precision، Recall و F1-score بررسی می‌شود. قطعه کد زیر نحوه‌ی اجرا در پایتون را نشان می‌دهد که برای پیاده‌سازی مدل، از کلاس SVC موجود در کتابخانه‌ی sklearn استفاده شده است

```
from sklearn.svm import SVC
clf_svc = SVC()
clf_svc.fit(X_train, y_train)
# Prediction on test data
y_pred_svc = clf_svc.predict(X_test)
# Accuracy Metrics
clf_metrics(y_test, y_pred_svc)
```

```
Results

Classification Accuracy Score: 0.6217008797653959

 Classification Report:
```

	precision	recall	f1-score	support
F-MOUTHBAR	0.00	0.00	0.00	87
F-TIDAL BAR	0.50	0.87	0.64	119
F-TIDAL CHANNEL	0.80	0.81	0.80	135
accuracy			0.62	341
macro avg	0.43	0.56	0.48	341
weighted avg	0.49	0.62	0.54	341

[9] Support Vector Classifier (SVC)

[10] Support Vector Regression (SVR)

نتایج نشـان می‌دهند که **دقت کلی مدل حدود ۶۲ درصـد** اسـت. با این‌حال، بررسـی معیارهای ارزیابی هر کلاس نشـان می‌دهد که عملکرد مدل در همه‌ی کلاس‌ها یکسـان نیسـت. به‌طور مشـخص، مدل نتوانسـته اسـت نمونه‌های کلاس "F-MOUTHBAR" را به‌درستی شناسایی کند؛ به‌طوری‌که مقادیر Precision، Recall و F1-score این کلاس همگی برابر با صفر هستند (در کد فوق، با رنگ سبز مشخص شده‌اند).

این نتیجه نشـان می‌دهد که ارزیابی مدل صـرفاً بر اسـاس دقت کلی، به‌ویژه در مسـائل دارای عدم توازن کلاس‌ها[11]، کافی نیست. در چنین شرایطی، ممکن است مدل در پیش‌بینی کلاس‌های پرتعداد عملکرد مناسبی داشته باشد، اما در شـناسـایی کلاس‌های کم‌نمونه ضـعیف عمل کند. برای بهبود عملکرد در این شـرایط، می‌توان از روش‌هایی مانند تنظیم وزن کلاس‌ها در فرایند آموزش، بازنمونه‌گیری داده‌ها و سـایر روش‌های مقابله با عدم توازن کلاس‌ها اسـتفاده کرد. در جدول ۱، مقایسه‌ی مدلهای مختلف برای مسئله‌ی فوق نشان داده شده است.

جدول ۱ امتیاز F1 مدل‌های طبقه‌بندی

مدل رگرسیونی	امتیاز F1
رگرسیون لجستیک چندجمله‌ای	0.648
الگوریتم طبقه‌بندی بردار پشتیبان	0.621

رگرسیون بردار پشتیبان

رگرسـیون بردار پشـتیبان، نسـخه‌ی رگرسـیونی ماشـین بردار پشـتیبان اسـت که برای پیش‌بینی مقادیر پیوسـته به کار می‌رود.

در این مثال، از کرنل تابع پایه‌ی شـعاعی اسـتفاده می‌کنیم تا روابط غیرخطی احتمالی بین متغیرهای ورودی و متغیر هدف مدل‌سـازی شـوند (این مسـئله در آموزش رگرسـیون خطی چندگانه معرفی شـده اسـت). قطعه کد زیر نحوه‌ی اجرا در پایتون را نشـان می‌دهد که برای پیاده‌سـازی مدل، از کلاس SVR موجود در کتابخانه‌ی sklearn استفاده شده است.

[11] Class Imbalance

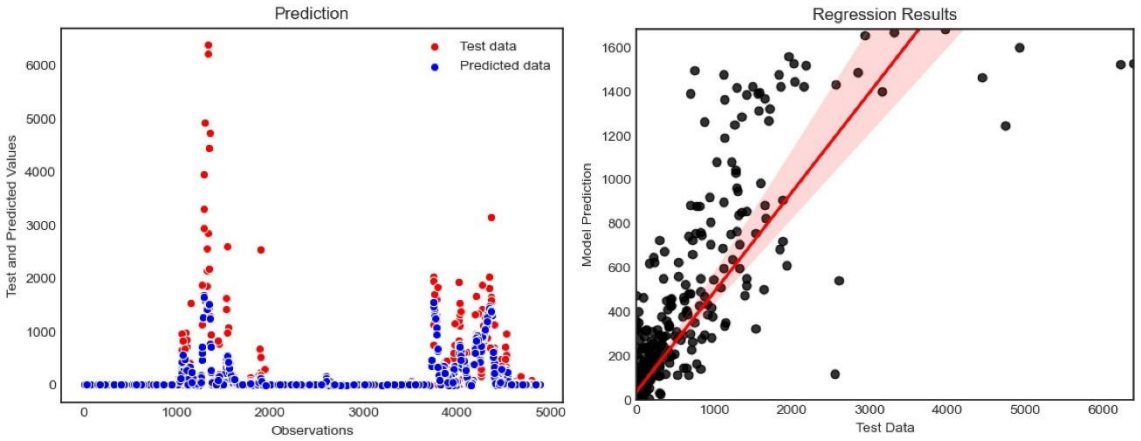
```
# Support Vector Regression (SVR)
from sklearn.svm import SVR
svr_reg = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=.1)
svr_reg.fit(X_train, y_train)
# Prediction on test data
y_pred_svr = svr_reg.predict(X_test)

Results
Mean Absolute Error: 82.06389051208197
Mean Squared Error: 112564.26261124818
Root Mean Squared Error: 335.5059799932755
R Squared: 0.6092197302512246
```

در قطعه کد فوق، نتایج ارزیابی مدل با رنگ زرد مشخص شده‌اند.

نمودارهای رگرسیون و پراکندگی نتایج حاصل از اجرای SVR در شکل ۸ نمایش داده شده‌اند. همان‌طور که مشاهده می‌شود، این مدل عملکرد بهتری نسبت به رگرسیون خطی چندگانه ارائه کرده است (

جدول ۲). این نتیجه نشان می‌دهد که SVR با استفاده از کرنل RBF توانسته است الگوهای موجود در داده‌ها را بهتر از مدل خطی موردنظر بازنمایی کند.



شکل ۸ : نتایج رگرسیون بردار پشتیبان بصورت نمودارهای رگرسیون و پراکندگی

جدول ۲ مقایسه‌ی مدلهای رگرسیون

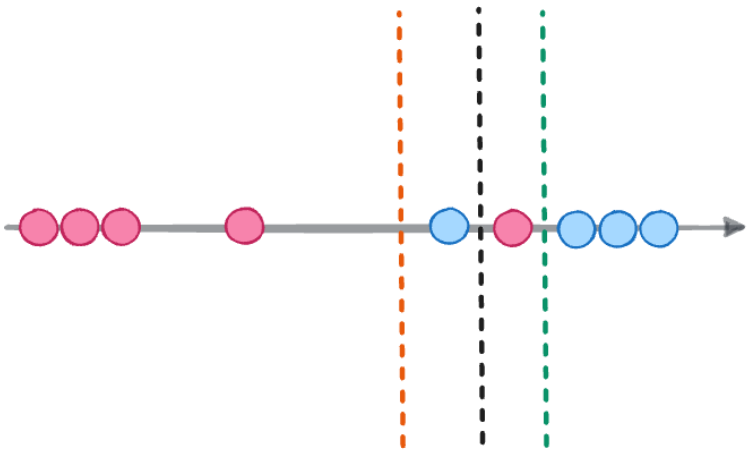
مدل	مقدار RMSE	مقدار R^2
رگرسیون خطی چندگانه	495.16	0.15
رگرسیون بردار پشتیبان	335.51	0.61

جمع‌بندی و نکات تکمیلی

۱- حاشیه‌ی سخت و حاشیه‌ی نرم چیست؟

در حالت ایده‌آل، ممکن است بتوان داده‌های دو کلاس را به‌طور کامل با یک مرز تصمیم خطی از یکدیگر جدا کرد (شکل ۳). در چنین شرایطی، می‌توان مرزی را انتخاب کرد که حاشیه‌ی جداسازی را بیشینه کند. به این رویکرد، حاشیه‌ی سخت[۱۲] گفته می‌شود.

اما در بسیاری از مسائل واقعی، داده‌ها به‌گونه‌ای هستند که نمی‌توان آن‌ها را به‌طور کامل با یک مرز خطی جدا کرد. برای مثال، همان‌طور که در شکل زیر مشاهده می‌کنید، صرف‌نظر از محل قرارگیری مرز تصمیم، همواره تعدادی از داده‌ها در سمت نامناسب مرز قرار می‌گیرند یا امکان جداسازی کامل آن‌ها وجود ندارد.



در چنین شرایطی، می‌توان به مدل اجازه داد که برخی از داده‌ها، حاشیه‌ی تعیین‌شده را نقض کنند و حتی در مواردی آن‌ها را به‌اشتباه طبقه‌بندی کند. این رویکرد، حاشیه‌ی نرم[۱۳] نامیده می‌شود و مبنای طبقه‌بند بردار پشتیبان (SVC) است. در مقابل، در روش حاشیه‌ی سخت، نقض محدودیت‌های جداسازی مجاز نیست.

۲- نقش بردارهای پشتیبان (Support Vectors)

همان‌طور که در شکل‌های پیشین مشاهده کردید، همه‌ی داده‌های آموزشی به یک اندازه در تعیین مرز تصمیم نقش ندارند. داده‌هایی که روی حاشیه یا در ناحیه‌ی نقض آن قرار می‌گیرند، می‌توانند در تعیین مرز تصمیم نقش داشته

---

[12] Hard Margin

[13] Soft Margin

باشند و بردارهای پشتیبان[14] نامیده می‌شوند. در مقابل، داده‌هایی که از حاشیه فاصله‌ی کافی دارند، معمولاً تأثیری مستقیم بر مرز تصمیم نهایی ندارند.

در خروجی ارزیابی مدل SVC در بخش اجرای عملی نیز ستونی با عنوان "support" مشاهده می‌شود که در قطعه کد مربوطه با رنگ زرد مشخص شده است. این ستون تعداد نمونه‌های هر کلاس را در مجموعه‌ی آموزشی که به‌عنوان بردار پشتیبان انتخاب شده‌اند، نشان می‌دهد. بنابراین، این مقدار تعداد کل نمونه‌های آن کلاس در مجموعه‌داده نیست، بلکه تعداد بردارهای پشتیبان مربوط به آن کلاس است.

۳- پارامترهای مهم در SVM

در الگوریتم ماشین بردار پشتیبان، تعیین مرز تصمیم مناسب را می‌توان به‌صورت یک مسئله‌ی بهینه‌سازی ریاضی بیان کرد. هدف در طبقه‌بندی، یافتن مرزی است که ضمن جداسازی مناسب کلاس‌ها، حاشیه‌ی جداسازی را تا حد امکان بزرگ کند. بااین‌حال، در مسائل واقعی همیشه امکان جداسازی کامل داده‌ها وجود ندارد؛ بنابراین، باید بین بزرگ‌بودن حاشیه و میزان خطا یا نقض محدودیت‌های جداسازی تعادل برقرار کرد.

برای بیان این مسئله، فرض کنید مجموعه‌ای شامل (n) نمونه‌ی آموزشی داریم. هر نمونه با بردار ویژگی $(\mathbf{x}_i)$ و برچسب کلاس (y_i) مشخص می‌شود؛ به‌طوری‌که در طبقه‌بندی دودویی، (y_i) یکی از مقادیر 1+ یا 1- است. مرز تصمیم خطی را می‌توان به‌صورت زیر نوشت:

$$\boldsymbol{\beta}^T\mathbf{x} + \beta_0 = 0$$

در این رابطه، $(\boldsymbol{\beta})$ ضرایب مربوط به ویژگی‌ها و (β_0) عرض از مبدأ است.

در حالت حاشیه‌ی نرم، متغیرهایی به نام ξ_i (که در برخی منابع با نماد (ϵ_i) نمایش داده می‌شوند) برای اندازه‌گیری میزان نقض محدودیت‌های حاشیه در نظر گرفته می‌شوند. مسئله‌ی بهینه‌سازی متداول برای SVM خطی به‌صورت زیر نوشته می‌شود:

$$\min_{\boldsymbol{\beta},\beta_0,\boldsymbol{\xi}} \quad \frac{1}{2}\|\boldsymbol{\beta}\|^2 + C\sum_{i=1}^{n}\xi_i$$

با رعایت محدودیت‌های زیر:

$$y_i(\boldsymbol{\beta}^T\mathbf{x}_i + \beta_0) \geq 1 - \xi_i \qquad i = 1, \dots, n$$
$$\xi_i \geq 0 \qquad i = 1, \dots, n$$

[14] Support Vectors

در این صورت‌بندی، جمله‌ی اول تابع هدف ($\min_{\boldsymbol{\beta},\beta_0,\boldsymbol{\xi}} \quad \frac{1}{2}\|\boldsymbol{\beta}\|^2$) به بزرگ‌بودن حاشیه کمک می‌کند و جمله‌ی دوم ($C\sum_{i=1}^{n}\xi_i$)، نقض محدودیت‌های حاشیه را جریمه می‌کند. پارامتر (C) میزان اهمیت این جریمه را تعیین می‌کند. بنابراین، انتخاب مقدار مناسب آن بر مرز تصمیم و عملکرد مدل تأثیر می‌گذارد.

پارامتر (C)

این پارامتر میزان جریمه‌ی نقض حاشیه و خطاهای آموزشی را کنترل می‌کند.

- مقدار بزرگ‌تر C: مدل نقض محدودیت‌ها را پرهزینه‌تر در نظر می‌گیرد و معمولاً تلاش می‌کند خطاهای آموزشی کمتری داشته باشد. این موضوع ممکن است به حاشیه‌ی کوچک‌تر و حساسیت بیشتر به داده‌های آموزشی منجر شود.
- مقدار کوچک‌تر C: مدل نقض محدودیت‌ها را آسان‌تر می‌پذیرد و معمولاً حاشیه‌ی بزرگ‌تری را ترجیح می‌دهد. البته اگر مقدار آن بیش از حد کوچک باشد، ممکن است مدل نتواند الگوهای داده‌ها را به‌اندازه‌ی کافی یاد بگیرد.

پارامتر (ϵ) در SVR

در رگرسیون بردار پشتیبان، پارامتر (ϵ) نقش متفاوتی دارد. این پارامتر پهنای ناحیه‌ی خطای قابل‌قبول پیرامون پیش‌بینی مدل را تعیین می‌کند. اگر اختلاف مقدار واقعی و پیش‌بینی‌شده در محدوده‌ی این ناحیه باشد، خطای مربوطه در تابع هدف استاندارد SVR جریمه نمی‌شود.

به‌صورت ساده، برای یک نمونه با مقدار واقعی (y_i) و مقدار پیش‌بینی‌شده‌ی ($\hat{y}_i$)، خطای خارج از این ناحیه را می‌توان به‌شکل زیر بیان کرد:

$$L_\epsilon(y_i, \hat{y}_i) = \max(0, |y_i - \hat{y}_i| - \epsilon)$$

بنابراین، افزایش (ϵ) ناحیه‌ی خطای قابل‌قبول را گسترده‌تر می‌کند و معمولاً تعداد نمونه‌هایی را که در تابع هدف جریمه می‌شوند کاهش می‌دهد. در مقابل، مقدار کوچک‌تر (ϵ) مدل را نسبت به اختلاف‌های کوچک‌تر حساس‌تر می‌کند.

برای درک بهتر نقش پارامتر (ϵ)، فرض کنید مقدار آن برابر با ۲ باشد. در این صورت، خطاهای کوچک‌تر یا مساوی ۲ جریمه نمی‌شوند و تنها بخشی از خطا که از این محدوده فراتر می‌رود، در تابع زیان محاسبه می‌شود. برای مثال، اگر اختلاف مقدار واقعی و پیش‌بینی‌شده برابر با ۱ باشد، مقدار زیان صفر خواهد بود:

$$L_\epsilon = \max(0, |1| - 2) = 0$$

اما اگر این اختلاف برابر با ۵ باشد، مقدار زیان برابر با ۳ خواهد بود:

$$L_\epsilon = \max(0, |5| - 2) = 3$$

بنابراین، تابع زیان اپسیلون‌حساس[15]، خطاهای داخل محدوده‌ی قابل‌قبول را نادیده می‌گیرد و تنها به بخش اضافی خطا جریمه اختصاص می‌دهد. همچنین، به دلیل استفاده از قدر مطلق، علامت خطا اهمیتی ندارد و اندازه‌ی آن تعیین‌کننده است.

پارامتر (γ) در کرنل RBF

در کرنل تابع پایه‌ی شعاعی(RBF) ، پارامتر γ میزان اثرگذاری هر نمونه بر نقاط دیگر را کنترل می‌کند. رابطه‌ی این کرنل به‌صورت زیر است:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

- مقدار بزرگ‌تر γ : اثر هر نمونه به ناحیه‌ی کوچک‌تری محدود می‌شود و مدل می‌تواند مرز تصمیم پیچیده‌تری ایجاد کند.
- مقدار کوچک‌تر γ : اثر نمونه‌ها گسترده‌تر می‌شود و مرز تصمیم معمولاً هموارتر خواهد بود.

انتخاب مناسب γ به ساختار داده‌ها و مقیاس ویژگی‌ها بستگی دارد؛ مقادیر نامناسب آن می‌تواند به بیش‌برازش یا کم‌برازش منجر شود.

٤- ترفند کرنل[16]

در این قسمت، با استفاده از کرنل چندجمله‌ای، نحوه‌ی عملکرد ترفند کرنل را بررسی می‌کنیم. ایده‌ی اصلی این است که بتوانیم ضرب داخلی داده‌ها را در یک فضای ویژگی با ابعاد بالاتر محاسبه کنیم، بدون آنکه لازم باشد مختصات داده‌ها را به‌صورت صریح در آن فضا محاسبه کنیم.

فرم کلی کرنل چندجمله‌ای به‌صورت زیر است:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + r)^d$$

---

[15] Epsilon-Insensitive Loss

[16] Kernel Trick

در این رابطه، $\mathbf{x}$و $\mathbf{z}$ دو بردار ورودی، (r) یک ثابت و (d) درجه‌ی چندجمله‌ای است.

برای درک بهتر این رابطه، ابتدا به یادآوری مفهوم ضـرب داخلی در جبر خطی توجه کنید. ضـرب داخلی دو بردار با ضرب مؤلفه‌های متناظر و جمع حاصل‌ضرب‌ها محاسبه می‌شود.

حال فرض کنید بخواهیم ضـرب داخلی دو داده را پس از نگاشـت آن‌ها به یک فضـای سـه‌بعدی محاسـبه کنیم. برای سـادگی، دو داده‌ی اولیه را با (a) و (b) نشـان می‌دهیم و نگاشـت آن‌ها در فضـای سـه‌بعدی را به‌صـورت زیر تعریف می‌کنیم:

$$\phi(a) = \left(a, a^2, \frac{1}{2}\right)$$
$$\phi(b) = \left(b, b^2, \frac{1}{2}\right)$$

ضرب داخلی این دو بردار در فضای سه‌بعدی برابر است با:

$$\phi(a)\phi(b) = ab + a^2b^2 + \frac{1}{4} = a^2b^2 + ab + \frac{1}{4} = \left(ab + \frac{1}{2}\right)^2$$

بنابراین، به‌جای آنکه ابتدا داده‌ها را به فضـای سـه‌بعدی نگاشـت کنیم و سـپس ضـرب داخلی آن‌ها را محاسـبه کنیم، می‌توانیم مستقیماً از رابطه‌ی زیر استفاده کنیم:

$$K(a, b) = \left(ab + \frac{1}{2}\right)^2$$

این همان ایده‌ی اصـلی ترفند کرنل اسـت؛ **محاسـبه‌ی ضـرب داخلی در فضـای ویژگی‌های با ابعاد بالاتر، بدون نیاز به محاسبه‌ی صریح مختصات داده‌ها در آن فضا.**

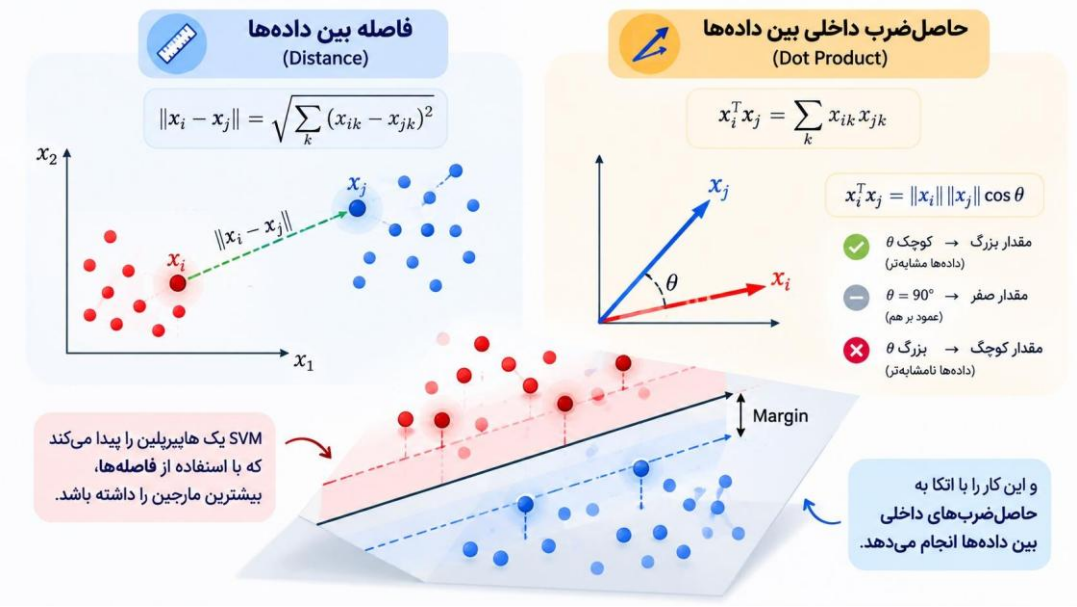
در حالت کلی، با انتخاب درجه‌ی (d) و ثابت(r) ، کرنل چندجمله‌ای به‌شکل زیر نوشته می‌شود:

$$K(a, b) = (ab + r)^d$$

در مسائل چندمتغیره نیز می‌توان از فرم برداری همین رابطه استفاده کرد:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + r)^d$$

در الگوریتم SVM ، استفاده از چنین کرنل‌هایی امکان مدل‌سازی مرزهای تصمیم غیرخطی را در فضای اولیه فراهم می‌کند، بدون آنکه لازم باشد نگاشت داده‌ها به فضای ویژگی به‌صورت صریح انجام شود.



۵- بهینه‌سازی هایپرپارامترها[17]

عملکرد یک الگوریتم یادگیری ماشین، علاوه بر داده‌های آموزشی، به مقادیری وابسته است که پیش از آموزش مدل تعیین می‌شوند. به این مقادیر **هایپرپارامتر** گفته می‌شود. برای مثال، در الگوریتم SVM، پارامترهایی مانند (C)، نوع کرنل، درجه‌ی کرنل چندجمله‌ای و مقدار (γ) از جمله هایپرپارامترهای مدل هستند. انتخاب مقادیر مناسب برای این هایپرپارامترها می‌تواند بر عملکرد مدل تأثیر قابل‌توجهی داشته باشد. ازاین‌رو، به‌جای انتخاب دستی مقادیر، می‌توان از روش‌های جستجو و بهینه‌سازی برای یافتن ترکیبی مناسب استفاده کرد.

در این مثال، از کتابخانه‌ی Hyperopt برای بهینه‌سازی هایپرپارامترهای طبقه‌بند بردار پشتیبان استفاده می‌کنیم. در بخش اجرای عملی با استفاده از تنظیمات پیش‌فرض، لیتوتایپ‌ها طبقه‌بندی شدند. اکنون می‌خواهیم با استفاده ازHyperopt ، ترکیب مناسب‌تری از هایپرپارامترها را پیدا کنیم. این کتابخانه با استفاده از یک تابع هدف، ترکیب‌های مختلف مقادیر را بررسی می‌کند و تلاش می‌کند مقداری را پیدا کند که تابع هدف را بهینه کند.

در این مثال، برای ارزیابی هر ترکیب از هایپرپارامترها، از **اعتبارسنجی متقابل پنج‌بخشی**[18] استفاده می‌کنیم. در این روش، داده‌های آموزشی به پنج بخش تقسیم می‌شوند. در هر مرحله، چهار بخش برای آموزش و بخش باقی‌مانده برای اعتبارسنجی استفاده می‌شود. این فرایند پنج بار تکرار می‌شود تا هر بخش یک بار نقش

---

[17] Hyperparameter Optimization

[18] 5-Fold Cross-Validation

مجموعه‌ی اعتبارسنجی را داشته باشد. سپس میانگین دقت حاصل از این پنج مرحله به‌عنوان معیار ارزیابی ترکیب موردنظر در نظر گرفته می‌شود.

در فرایند بهینه‌سازی، سه مفهوم اصلی داریم:

- **فضای جستجو (Search Space)** : مجموعه‌ی مقادیر یا گزینه‌هایی که هایپرپارامترها می‌توانند اختیار کنند؛ برای مثال، انواع کرنل و محدوده‌ی مقادیر C و γ.
- **تابع هدف (Objective Function)** : تابعی که کیفیت هر ترکیب از هایپرپارامترها را ارزیابی می‌کند. در این مثال، میانگین دقت حاصل از اعتبارسنجی متقابل محاسبه می‌شود.
- **الگوریتم جستجو** : روشی که تعیین می‌کند در هر مرحله کدام ترکیب از هایپرپارامترها بررسی شود. در این مثال، از روش TPE در کتابخانه‌ی Hyperopt استفاده می‌کنیم.

در ادامه، تابع هدف و فضای جستجو تعریف می‌شوند .تابع cross_val_score میانگین دقت مدل را در اعتبارسنجی متقابل محاسبه می‌کند. از آنجا که Hyperopt به‌دنبال کمینه‌کردن تابع هدف است، مقدار منفی میانگین دقت را برمی‌گردانیم تا کمینه‌سازی آن معادل بیشینه‌سازی دقت باشد.

```
def objective(params):
    svc = SVC(**params)
    return -1. * cross_val_score(svc, X_train, y_train).mean()
kernels = ['rbf','poly','linear','sigmoid']
space = {'C':hp.lognormal('C', 0, 1), 'kernel':hp.choice('kernel',
                                                         kernels),
         'degree':hp.choice('degree', range(1, 15)),
         'gamma':hp.uniform('gamma', 1e-2, 1e2)
}
trials = Trials()
best_svc = fmin(objective, space, algo=tpe.suggest, max_ evals=50,
                trials=trials)
print(best_svc)
{'C': 1751.5444736790396, 'degree': 11, 'gamma': 5.899864955354083,
 'kernel': 0}
```

خروجی بهینه‌سازی در این اجرا به‌صورت زیر است:

```
{
    'C': 1751.5444736790396,
    'degree': 11,
    'gamma': 5.899864955354083,
    'kernel': 0
}
```

تفسیر خروجی: مقدارهای به‌دست‌آمده، ترکیب منتخب الگوریتم جستجو هستند. در این خروجی، مقدار kernel برابر صفر است؛ زیرا hp.choice برای گزینه‌های انتخابی یک شاخص برمی‌گرداند. بنابراین، با توجه به فهرست تعریف‌شده (در کد فوق با رنگ زرد مشخص شده است)، kernels[0] برابر 'rbf' است.

اکنون مدل را با هایپرپارامترهای انتخاب‌شده آموزش می‌دهیم و عملکرد آن را روی داده‌های آزمون ارزیابی می‌کنیم.

```
svc = SVC(
    C=best_svc['C'],
    kernel=kernels[best_svc['kernel']],
    degree=best_svc['degree'],
    gamma=best_svc['gamma']
)

svc.fit(X_train, y_train)

# Prediction on test data
y_pred = svc.predict(X_test)

# Accuracy Metrics
clf_metrics(y_test, y_pred)
Classification Accuracy Score: 0.9618768328445748
```

نتایج این مثال نشان می‌دهند که دقت مدل در این اجرا، از حدود (۶۲٪) با تنظیمات پیش‌فرض به حدود (۹۶.۲٪) پس از بهینه‌سازی افزایش یافته است. این بهبود نشان می‌دهد که انتخاب مناسب هایپرپارامترها می‌تواند نقش مهمی در عملکرد الگوریتم‌های یادگیری ماشین داشته باشد.

البته این نتیجه به مجموعه‌داده، نحوه‌ی تقسیم داده‌ها و فضای جستجوی تعریف‌شده وابسته است و تضمین نمی‌کند که بهینه‌سازی در همه‌ی مسائل چنین افزایشی در دقت ایجاد کند. همین رویکرد را می‌توان برای بهینه‌سازی هایپرپارامترهای بسیاری از الگوریتم‌های دیگر یادگیری ماشین و یادگیری عمیق نیز به کار برد.